

title: "Process Modeling Best Practices" tags: [Process Modeling, BPMN, Best Practices, Business Process]
summary: "Guidance on creating clear, consistent, and effective process models for business analysts and architects."

Process Modeling Best Practices

Creating effective process models is about clarity and consistency. A well-crafted process diagram should be **easy to understand** by any stakeholder, whether business or technical ¹. To achieve this, modelers must apply proven standards: explicit start/end events, standardized symbols, and a logical flow that avoids tangled "spaghetti" lines ² ³. This handbook gathers decades of best-practice wisdom, combining formal BPMN conventions with practical, visual design guidance.

Modeling Clarity Fundamentals

- **Avoid Spaghetti Flows.** Process diagrams that are too interconnected become confusing. If flows cross or loop unpredictably, readers lose the thread ² ⁴. Break large or complex processes into sub-processes to minimize crossings. Keep sequence flows mostly left-to-right, and use explicit gateways (e.g. AND/OR diamonds) to structure forks and joins. As Trisotech warns, diagrams with "long, meandering, crossing lines" obscure the logic ⁴. Always refactor very large models: if the longest path is huge or contains many joins/splits, consider decomposing it ² ⁵.
- **Maintain Consistency.** Consistent notation and naming make models immediately understandable. Use the same symbol (shape and color) for the same concept everywhere ⁶. Define or adopt a standard palette of BPMN elements (tasks, events, gateways, etc.) so a reader can interpret any diagram without a legend ⁷ ⁸. Consistency extends to naming: pick a style (e.g. verb-object for activities) and apply it uniformly ⁹ ¹⁰. Without this consistency, even valid models "become confusing" and defeat their purpose ⁶.
- **Explicit Start and End.** Every process model needs a clear beginning and termination. In BPMN, always include a **start event** and one or more **end events** ¹¹ ¹². The start event triggers the first activity; without it, the process cannot be executed ¹³. Likewise, explicitly define the end event to mark the business goal or completion state ¹². If there are alternative end states (success vs failure), use separate end events. Trisotech notes that processes with "implicit start and end events" are undesirable ¹⁴. Always label start/end meaningfully (e.g. "Order Received" start, "Order Completed" end) to make the process's scope clear ¹⁵ ³.
- **Balance Breadth vs. Depth.** Aim for a high-level view that is comprehensive but not overloaded, then drill down for detail. Top-level diagrams should fit on one page and show the entire end-to-end flow ¹⁶. Use sub-processes or call-activities to encapsulate detail and keep the main path clean ¹⁷ ⁵. For example, an order-to-cash process may show "Process Order" as a single box at level 1, with

a collapsed subprocess icon. Drill into that subprocess separately to reveal invoice, shipment, and payment tasks. This hierarchical approach lets stakeholders focus on the primary flow, while detailed flows are accessible on demand. The modeling convention is to build a **multi-layer hierarchy**: one master process, with subprocesses for phases, and further breakdowns if needed ⁵. Avoid overcrowding a single diagram with every detail; instead, explicitly link to child diagrams for depth

¹⁶ ¹⁷ .

- **Unambiguous Roles & Responsibilities.** Clearly assign who does what. Use pools and lanes to delineate roles, departments, or systems ¹⁸. For example, a pool could represent an organization, with lanes for “Sales”, “Finance”, and “Warehouse”. Each task is placed in the lane of its responsible role. Label each lane with the role name (e.g. “*Purchasing Dept.*” or “*Customer Service*”) ¹⁹. Do not overload a lane with multiple roles or leave tasks floating outside lanes. Properly defined pools/lanes ensure that the workflow is clearly partitioned and each step’s owner is evident ¹⁸ ¹⁹. If collaborating organizations interact, use message flows between pools; do not try to show multi-actor work all within one lane. Clarity about responsibilities reduces handoff confusion and makes ownership explicit.

Visual Communication Techniques

- **Strategic Use of Color and Highlights.** Thoughtful color usage can guide the eye but must be consistent. For example, you might color all “approval” tasks in one hue and “notification” tasks in another. Use a consistent palette across diagrams so that the same color always means the same type of activity. Cognitive research shows that consistent color schemes and clear visual grouping **reduce cognitive load** and help viewers navigate complex models ²⁰. Use bold shapes or colored highlights to emphasize key flows or decision points when presenting. For instance, highlight the critical “happy path” or exception route with a light shading. Do **not** rely on color alone to convey meaning (since some stakeholders may print in grayscale or be colorblind); always accompany color with clear labels or legend notes. In summary, a consistent, high-contrast design (e.g. bold outlines, uniform stroke weights) aids quick perception ²⁰.
- **Annotations and Documentation.** Use text annotations wisely. If a gateway’s logic is not obvious, attach a text annotation explaining it ²¹. For example, next to a decision diamond, an annotation might clarify the business rule (“If customer is preferred, skip credit check”). But avoid cluttering the diagram with long prose. Key decision points or unusual terms should be annotated. If a BPMN element has an implicit detail (like a timer interval), consider annotating it in text for human readers. A good rule: **almost everything should have a meaningful label** ²². Do not leave default names (e.g. “Task 1”) — rename every task, event, and gateway. This turns the model into a self-explanatory narrative.
- **Presenting Subprocesses Clearly.** When a subprocess is used, show it as a single collapsed activity box with a “+” (in BPMN) or simply a labeled group (in a flowchart), and clearly link it to its detailed view. In big models, use the same name and context so viewers know the subprocess’s place in the big picture ¹⁷ ⁵. During workshops, explain that detailed subprocess contents are in a separate diagram. On the main view, treat the subprocess box as an atomic step in the sequence flow. Avoid drawing all the internals of a subprocess on the top-level model – that is precisely why the subprocess exists. This keeps the **main thread** obvious.

- **Visual Clarity in Workshops.** In collaborative sessions, clarity is king. Use a clean layout: keep flows mostly left-to-right, minimize crossed lines, and leave whitespace around elements ²³ ²⁰ . Build the model iteratively with stakeholders to ensure understanding: ask if the left-right order makes sense, if parallel paths are explicit, and whether the “happy path” is obvious. Large font and simple shapes help readability. For presentations, progressively reveal the model: focus on one swimlane or subprocess at a time. Use color or animation (if digital) to follow the flow step-by-step. According to cognitive guidelines, highlighting critical paths (for example, with a bold outline or different color) directs attention to the key sequence ²⁰ . Always double-check with stakeholders: can they describe the process back using the diagram alone? Workshops often uncover missing steps or misnamed tasks; treat these as refinements, not mistakes.

Top 20 Pitfalls & Fixes

The following examples illustrate common modeling pitfalls (left diagrams) and their fixes (right diagrams). The first 10 use **Mermaid** syntax (an easy text-based flowchart), and the next 10 use **BPMN 2.0 XML** syntax. Each pair shows “Before” (the problematic model) and “After” (the corrected model).

Mermaid Examples (Pitfalls 1–10)

1. **Unstructured, crossing flows (Spaghetti).** In the “before” diagram, flows cross and loop, making the process hard to follow. The fix is to reorder and group steps, reducing crossings and clarifying the path.

```
flowchart LR
  A[Start onboarding] --> C[Check background]
  A --> B[Collect documents]
  C --> D[Schedule orientation]
  B --> D
  D --> E[End onboarding]
```

Improved (flows untangled, tasks in logical order):

```
flowchart LR
  A[Start onboarding] --> B[Collect documents]
  B --> C[Check background]
  C --> D[Schedule orientation]
  D --> E[End onboarding]
```

2. **Missing explicit start/end.** The “before” model begins with a task and no clear start symbol; it ends abruptly. Every process needs a designated start and end. We add a distinct “Start” node and an “End” node to make entry/exit points clear.

```
flowchart LR
  A[Perform health check] --> B[Archive report]
```

Improved (added start and end events):

```
flowchart LR
  S((Start)) --> A[Perform health check]
  A --> B[Archive report]
  B --> E((End))
```

3. **Inconsistent naming.** In the "before" diagram, one node is in all-caps (PAY FEES) while others are sentence case. Inconsistency distracts readers. The fix is to standardize to sentence-case verb-object labels.

```
flowchart LR
  A[Receive application] --> B[PAY FEES]
  B --> C[Issue approval]
```

Improved (consistent casing and phrasing):

```
flowchart LR
  A[Receive application] --> B[Pay fees]
  B --> C[Issue approval]
```

4. **Too much detail at once.** The "before" model lists every tiny step inline, making it large and busy. Instead, collapse sub-steps into a single higher-level task (or use subgraphs) to balance breadth vs. depth. Here we abstract the multi-step approval process.

```
flowchart LR
  A[Submit report] --> B[Manager review]
  B --> C[Finance review]
  C --> D[Director sign-off]
  D --> E[Send finalized report]
```

Improved (abstract detailed approvals into one subprocess):

```
flowchart LR
  A[Submit report] --> F{Approve report?}
  F -->|Yes| E[Send finalized report]
```

```
F -->|No| G[Revise report]
G --> F
```

5. **Unlabeled gateway (decision) paths.** The "before" flow forks but doesn't explain on which conditions. All diverging paths should be labeled. In the fix, add clear yes/no labels on the outgoing branches of the decision.

```
flowchart LR
  A{In stock?} --> B[Pack order]
  A --> C[Notify customer]
```

Improved (decision labeled with branch conditions):

```
flowchart LR
  A{In stock?} -->|Yes| B[Pack order]
  A -->|No| C[Notify customer]
```

6. **Mixed abstraction levels.** The "before" has a mix of abstract and concrete tasks (e.g., "Prepare" vs "Review contract") in one flow. The fix is to keep levels consistent: either detail everything or abstract repeated logic. Here we replace an intermediate detail with a sub-process node.

```
flowchart LR
  A[Start project] --> B[Assign team]
  B --> C[Prepare budget]
  C --> D[Review contract]
  D --> E[Launch project]
```

Improved (combined related steps under a sub-process):

```
flowchart LR
  A[Start project] --> B[Assign team]
  B --> F[Finalize planning]
  F --> E[Launch project]
```

7. **No explicit join for parallel flows.** In the "before", two branches reconverge at the same task without a gateway, which can be misinterpreted. The fix introduces a merge node or an explicit merge indication so it's clear that both branches join before continuing.

```
flowchart LR
  A --> B[Task 1]
  A --> C[Task 2]
```

```
B --> D[Finalize]
C --> D
```

Improved (insert a merge point):

```
flowchart LR
  A --> G((Merge))
  G --> B[Task 1]
  G --> C[Task 2]
  B --> D[Finalize]
  C --> D
```

8. **Overlapping flows.** The "before" diagram has a criss-cross where the flow from C to D crosses A's path. Such layouts confuse the direction of flow. The fix is to reorder or bend lines to eliminate crossings.

```
flowchart LR
  A[Start] --> B[Step 1]
  D[Step 3] --> C[Step 2]
  B --> D
```

Improved (straightened flow direction):

```
flowchart LR
  A[Start] --> B[Step 1]
  B --> C[Step 2]
  C --> D[Step 3]
```

9. **Tiny or unclear tasks.** The "before" has a task named vaguely (Check). It's unclear what is being checked. In the fix, rename it to be specific and consistent with others.

```
flowchart LR
  A[Start] --> B[Check]
  B --> C[Finalize]
```

Improved (clear verb-object labels):

```
flowchart LR
  A[Start] --> B[Check data integrity]
  B --> C[Finalize report]
```

10. **Omitting a required end.** The "before" has tasks and flows but no terminal node. The fix is to always end with an explicit end event (double circle).

```
flowchart LR
    A[Receive ticket] --> B[Resolve issue]
```

Improved (added end event):

```
flowchart LR
    A[Receive ticket] --> B[Resolve issue]
    B --> C((End))
```

BPMN Examples (Pitfalls 11–20)

1. **Missing Start/End events.** The process below omits both. In BPMN, a process with implicit start/end is confusing. We fix it by adding `<startEvent>` and `<endEvent>` elements.

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc1" isExecutable="false">
    <task id="task1" name="Review Request" />
    <task id="task2" name="Approve Request" />
    <sequenceFlow id="flow1" sourceRef="task1" targetRef="task2" />
  </process>
</definitions>
```

Improved (added start/end events and flows):

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc1" isExecutable="false">
    <startEvent id="start1" name="Request Submitted" />
    <task id="task1" name="Review Request" />
    <sequenceFlow id="flowA" sourceRef="start1" targetRef="task1"/>
    <task id="task2" name="Approve Request" />
    <sequenceFlow id="flowB" sourceRef="task1" targetRef="task2"/>
    <endEvent id="end1" name="Request Complete"/>
    <sequenceFlow id="flowC" sourceRef="task2" targetRef="end1"/>
  </process>
</definitions>
```

2. **Parallel flow without gateways.** In the "before", Task A splits into Task B and C simultaneously with no gateway. Proper BPMN requires a parallel gateway for clarity. We insert `<parallelGateway>`

split and merge.

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc2" isExecutable="false">
    <task id="task1" name="Task A"/>
    <task id="task2" name="Task B"/>
    <task id="task3" name="Task C"/>
    <sequenceFlow id="flow1" sourceRef="task1" targetRef="task2"/>
    <sequenceFlow id="flow2" sourceRef="task1" targetRef="task3"/>
  </process>
</definitions>
```

Improved (added parallel split and merge):

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc2" isExecutable="false">
    <task id="task1" name="Task A"/>
    <parallelGateway id="split"/>
    <sequenceFlow id="flow0" sourceRef="task1" targetRef="split"/>
    <task id="task2" name="Task B"/>
    <task id="task3" name="Task C"/>
    <sequenceFlow id="flow1" sourceRef="split" targetRef="task2"/>
    <sequenceFlow id="flow2" sourceRef="split" targetRef="task3"/>
    <parallelGateway id="merge"/>
    <sequenceFlow id="flow3" sourceRef="task2" targetRef="merge"/>
    <sequenceFlow id="flow4" sourceRef="task3" targetRef="merge"/>
    <task id="task4" name="Task D"/>
    <sequenceFlow id="flow5" sourceRef="merge" targetRef="task4"/>
    <endEvent id="end" name="End"/>
  </process>
</definitions>
```

3. **No gateway for conditional branch.** The "before" directly connects one task to two successors, but no gateway is shown. We add an `<exclusiveGateway>` to make the condition explicit.

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc3" isExecutable="false">
    <task id="t1" name="Check Inventory"/>
    <task id="t2" name="Process Order"/>
    <task id="t3" name="Cancel Order"/>
    <sequenceFlow id="flow1" sourceRef="t1" targetRef="t2"/>
```



```

    <sequenceFlow id="flow2" sourceRef="t1" targetRef="t3"/>
  </process>
</definitions>

```

Improved (inserted exclusive gateway with conditions):

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc3" isExecutable="false">
    <task id="t1" name="Check Inventory"/>
    <exclusiveGateway id="gw1" name="In Stock?"/>
    <sequenceFlow id="flow1" sourceRef="t1" targetRef="gw1"/>
    <task id="t2" name="Process Order"/>
    <task id="t3" name="Cancel Order"/>
    <sequenceFlow id="flow2" sourceRef="gw1" targetRef="t2"/>
    <sequenceFlow id="flow3" sourceRef="gw1" targetRef="t3"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

4. **Tasks not named meaningfully.** The "before" code has generic task names (Task1 etc.). Each activity should have a descriptive name. The fix is to rename tasks to indicate the business action.

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc4" isExecutable="false">
    <startEvent id="start"/>
    <task id="taskA" name="Task1"/>
    <task id="taskB" name="Task2"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="taskA"/>
    <sequenceFlow id="f2" sourceRef="taskA" targetRef="taskB"/>
  </process>
</definitions>

```

Improved (tasks renamed to verb-object style):

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc4" isExecutable="false">
    <startEvent id="start"/>
    <task id="taskA" name="Review Document"/>
    <task id="taskB" name="Archive Document"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="taskA"/>
    <sequenceFlow id="f2" sourceRef="taskA" targetRef="taskB"/>
  </process>
</definitions>

```

```

    <endEvent id="end"/>
  </process>
</definitions>

```

5. **Missing merge gateway for parallel paths.** Here two tasks converge into one without a gateway. In BPMN, parallel executions should be explicitly joined. We add a `<parallelGateway>` to merge them before continuing.

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc5" isExecutable="false">
    <task id="tA" name="Prepare reports"/>
    <task id="tB" name="Review data"/>
    <task id="tC" name="Publish results"/>
    <sequenceFlow id="fA" sourceRef="tA" targetRef="tC"/>
    <sequenceFlow id="fB" sourceRef="tB" targetRef="tC"/>
  </process>
</definitions>

```

Improved (added parallel merge gateway):

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc5" isExecutable="false">
    <task id="tA" name="Prepare reports"/>
    <task id="tB" name="Review data"/>
    <parallelGateway id="merge"/>
    <sequenceFlow id="fA" sourceRef="tA" targetRef="merge"/>
    <sequenceFlow id="fB" sourceRef="tB" targetRef="merge"/>
    <task id="tC" name="Publish results"/>
    <sequenceFlow id="fC" sourceRef="merge" targetRef="tC"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

6. **Single pool with no lanes.** The process has multiple roles but no lanes. To clarify responsibilities, we insert lanes for each role and assign tasks.

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc6" isExecutable="false">
    <startEvent id="start"/>
    <task id="t1" name="Draft email"/>

```

```

    <task id="t2" name="Send email"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="t1"/>
    <sequenceFlow id="f2" sourceRef="t1" targetRef="t2"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

Improved (added lanes for roles):

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc6" isExecutable="false">
    <laneSet>
      <lane id="lane1" name="Employee">
        <flowNodeRef>t1</flowNodeRef>
      </lane>
      <lane id="lane2" name="System">
        <flowNodeRef>t2</flowNodeRef>
      </lane>
    </laneSet>
    <startEvent id="start"/>
    <task id="t1" name="Draft email"/>
    <task id="t2" name="Send email"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="t1"/>
    <sequenceFlow id="f2" sourceRef="t1" targetRef="t2"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

7. **No annotation for non-obvious logic.** In BPMN, if a condition or detail isn't clear, add a `<textAnnotation>`. Here we add a note explaining the gateway decision.

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc7" isExecutable="false">
    <startEvent id="start"/>
    <exclusiveGateway id="gw" name="Credit check passed?"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="gw"/>
    <task id="tYes" name="Authorize purchase"/>
    <task id="tNo" name="Reject order"/>
    <sequenceFlow id="f2" sourceRef="gw" targetRef="tYes"/>
    <sequenceFlow id="f3" sourceRef="gw" targetRef="tNo"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

```

    </process>
</definitions>

```

Improved (added annotation):

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc7" isExecutable="false">
    <laneSet><lane id="l1" name="Sales"><flowNodeRef>gw</flowNodeRef></lane></laneSet>
    <startEvent id="start"/>
    <exclusiveGateway id="gw" name="Credit check passed?"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="gw"/>
    <task id="tYes" name="Authorize purchase"/>
    <task id="tNo" name="Reject order"/>
    <textAnnotation id="note1" text="If customer's credit rating is 650 or above"/>
    <association id="as1" sourceRef="note1" targetRef="gw"/>
    <sequenceFlow id="f2" sourceRef="gw" targetRef="tYes"/>
    <sequenceFlow id="f3" sourceRef="gw" targetRef="tNo"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

8. **Parallel gateways without pairing.** A split gateway should match a corresponding join. The "before" splits flow with `<parallelGateway>` but never merges them. The fix adds a second `<parallelGateway>` to synchronize threads before ending.

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc8" isExecutable="false">
    <startEvent id="start"/>
    <parallelGateway id="split"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="split"/>
    <task id="t1" name="Do A"/>
    <task id="t2" name="Do B"/>
    <sequenceFlow id="f2" sourceRef="split" targetRef="t1"/>
    <sequenceFlow id="f3" sourceRef="split" targetRef="t2"/>
    <!-- Missing merge gateway here -->
    <endEvent id="end"/>
  </process>
</definitions>

```

Improved (added merge gateway):

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc8" isExecutable="false">
    <startEvent id="start"/>
    <parallelGateway id="split"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="split"/>
    <task id="t1" name="Do A"/>
    <task id="t2" name="Do B"/>
    <sequenceFlow id="f2" sourceRef="split" targetRef="t1"/>
    <sequenceFlow id="f3" sourceRef="split" targetRef="t2"/>
    <parallelGateway id="merge"/>
    <sequenceFlow id="f4" sourceRef="t1" targetRef="merge"/>
    <sequenceFlow id="f5" sourceRef="t2" targetRef="merge"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

9. **Missing sequence flow labels on conditions.** The "before" has multiple outgoing flows but no conditions documented. We add `<conditionExpression>` on the sequence flows to clarify, or at least annotate the flows with text. (BPMN requires text or conditional flows on gateways.)

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc9" isExecutable="false">
    <exclusiveGateway id="g1" />
    <task id="tX" name="A"/>
    <task id="tY" name="B"/>
    <sequenceFlow id="f1" sourceRef="g1" targetRef="tX"/>
    <sequenceFlow id="f2" sourceRef="g1" targetRef="tY"/>
  </process>
</definitions>

```

Improved (condition expressions added to flows):

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc9" isExecutable="false">
    <exclusiveGateway id="g1" name="Decide path"/>
    <task id="tX" name="Path A"/>
    <task id="tY" name="Path B"/>
    <sequenceFlow id="f1" sourceRef="g1" targetRef="tX">
      <conditionExpression xsi:type="tFormalExpression">condition = A</conditionExpression>
    </sequenceFlow>
  </process>
</definitions>

```

```

    <sequenceFlow id="f2" sourceRef="g1" targetRef="tY">
      <conditionExpression xsi:type="tFormalExpression">condition = B</
conditionExpression>
    </sequenceFlow>
  <endEvent id="end"/>
</process>
</definitions>

```

10. **Textual clutter and formatting.** The "before" uses inconsistent capitalization and has extra whitespace inside tags. The fix standardizes label case and cleans formatting. While not affecting execution, tidy XML aids readability and maintenance.

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc10" isExecutable="false">
    <startEvent id="start"/>
    <task id="tA" name="SEND Invoice"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="tA"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

Improved (consistent naming, no extraneous spaces):

```

<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL">
  <process id="Proc10" isExecutable="false">
    <startEvent id="start"/>
    <task id="tA" name="Send Invoice"/>
    <sequenceFlow id="f1" sourceRef="start" targetRef="tA"/>
    <endEvent id="end"/>
  </process>
</definitions>

```

Each "before" and "after" example above demonstrates a common pitfall and its fix. Applying these tips — from structuring flows to annotating decisions — will make your process models **clearer, more consistent, and easier to communicate**.

1 6 7 8 BPMN Modeling Conventions: A Comprehensive Guide - SAP Signavio

<https://www.signavio.com/post/bpmn-modeling-conventions/>

2 Process Modeling Guidelines: Systematic Literature Review and Experiment

<https://lume.ufrgs.br/bitstream/handle/10183/175028/001063588.pdf>

3 4 5 11 14 16 23 **BPMN Modeling Best Practices**

<https://www.trisotech.com/bpmn-modeling-best-practices/>

9 19 21 22 **Naming Conventions for BPMN Diagrams**

<https://www.trisotech.com/naming-conventions-for-bpmn-diagrams/>

10 **Naming BPMN elements | Camunda 8 Docs**

<https://docs.camunda.io/docs/components/best-practices/modeling/naming-bpmn-elements/>

12 13 15 17 18 **Business Process Model & Notation - BPMN Introductory Guide | Signavio**

<https://www.signavio.com/bpmn-introductory-guide/>

20 **Cognitive Factors in Process Model Comprehension—A Systematic Literature Review**

<https://www.mdpi.com/2076-3425/15/5/505>